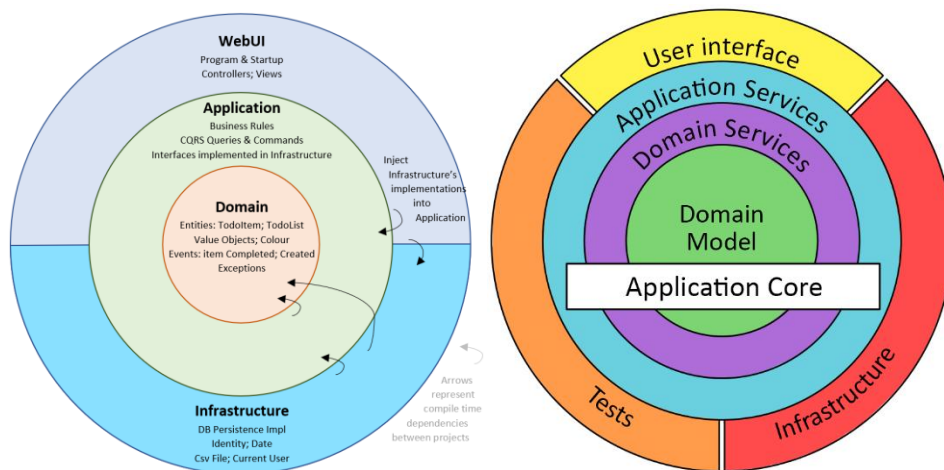


Clean Architecture

- Clean Architecture is a software architecture style introduced by Robert C. Martin (Uncle Bob). Its goal is to create **highly maintainable**, **testable**, and **independent** systems.
- Clean architecture is a software design approach that emphasizes the separation of concerns, making systems easier to maintain, test, and scale.
- It organizes software into distinct layers, each with a specific responsibility, ensuring that the core business logic remains independent of external frameworks, UI, databases, and delivery mechanisms
- Clean Architecture is a set of principles and a layered architecture pattern that helps you build systems that are maintainable, testable, independent of frameworks, and adaptable to change. The big idea is **separation of concerns + dependency rule**: inner layers know nothing about outer layers; outer layers depend on inner layers (via interfaces/abstractions).



What Clean Architecture Achieves

- **Independent of Frameworks**
The architecture does not depend on ASP.NET Core, EF Core, SQL Server, or any external tool.
- **Independent of UI**
You can switch from Web API → MVC → Blazor → gRPC without modifying domain logic.
- **Independent of Database**
You can replace SQL Server with PostgreSQL, Oracle, MongoDB, or an in-memory DB easily.

- **Testable**

Business rules can be unit-tested without infrastructure.

Clean Architecture — Layer Overview

A typical ASP.NET Core Clean Architecture solution has 4 layers:

1 Domain Layer (Core Business Logic)

- Contains entities, enums, value objects, domain events, interfaces (only if needed).
- No database code, no HTTP code.
- Pure C# classes.
 - **What:** Enterprise-wide business rules, core domain models (rich domain objects or simple POCOs).
 - **Responsibility:** Hold state and domain logic that won't change if UI or DB changes.
 - **Examples:** Order, Customer, domain value objects, domain exceptions, invariants.
 - **Characteristics:** Pure, testable, no framework dependencies.

2 Application Layer (Use Cases / Interactors / Application Services)

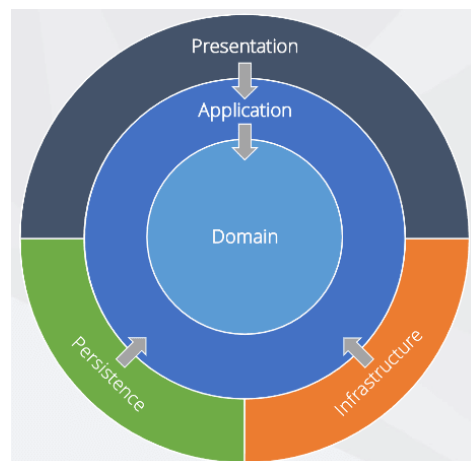
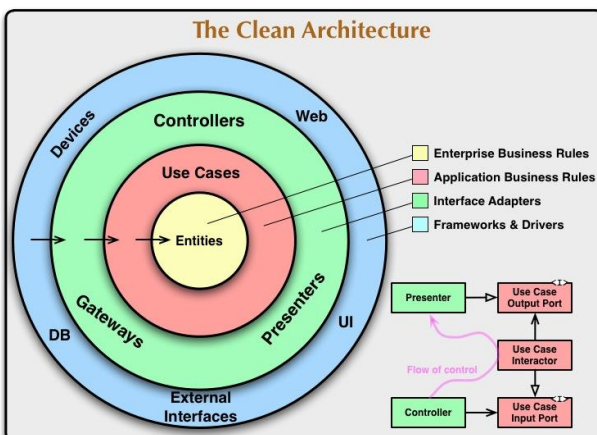
- **This is where business rules live. Contains:**
- Interfaces (Repository/Service contracts)(**No EF Core here** — only interfaces.)
- DTOs
- CQRS (Commands/Queries)
- Validators
- Services
 - **What:** Application-specific business rules. Orchestrates entities to accomplish application tasks.
 - **Responsibility:** Implement use cases (e.g., CreateOrder, CancelOrder, GetCustomerSummary), coordinate repositories/gateways, enforce application-level rules.
 - **Characteristics:** Knows interfaces (ports) for persistence, messaging, etc., but uses abstractions — not implementations.
 - **Testability:** Very high — inject fake repositories to test the use case.

3 Infrastructure Layer (Implementation of Interfaces)

- This layer talks to:
 - EF Core
 - SQL Server
 - Logging
 - Authentication
 - File storage
- It implements the interfaces defined in Application Layer/Domain Layer.
 - What: Concrete implementations: EF Core, SQL Server, File System, external APIs, ASP.NET Core framework glue.
 - Responsibility: Provide concrete implementations of interfaces defined by the inner layers (e.g., IOrderRepository), external integrations, configuration.
 - Placement: Outermost layer — infrastructure code depends on inner layers (interfaces) but not vice versa.

4 Presentation Layer (UI = ASP.NET Core Web API)

- Contains Controllers
- Minimal endpoints
- DI setup
- This layer **interacts with Application Layer**.
 - **What:** Translate between outer layer models (HTTP, DB rows, UI view models) and inner models (Entities, UseCase DTOs).
 - **Responsibility:** Convert requests/responses, implement interfaces required by use cases (repositories/gateways) and adapt them to infrastructure implementations.
 - **Common components:** Controllers (API layer), Presenters/ViewModels, Repository interfaces implementation skeletons that call EF/SQL.

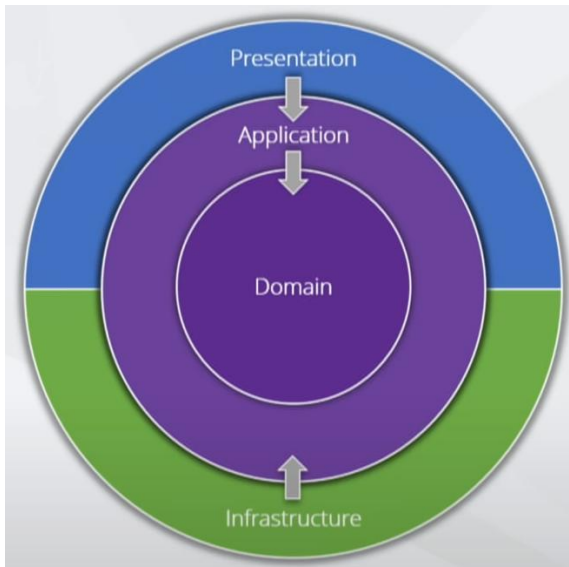


High-level goals

- Independence from frameworks — framework code (ASP.NET, EF Core) stays at the edges.
- Testability — core business rules are easy to unit-test (no HTTP, DB, or disk I/O).
- UI/Database/Infrastructure replaceability — change DB or UI without touching business rules.
- Clear boundaries — each layer has a single responsibility and clear API.

Clean Architecture Rule

-  Dependencies must always point inward.
- UI → Application → Domain
- Infrastructure → Application → Domain
- Domain should NOT depend on any other layer.



All dependencies must point inward.

Depends On	Does NOT depend on
Presentation → Application	Application → Presentation
Application → Domain	Domain → Application
Infrastructure → Application	Application → Infrastructure

💡 Why This Rule?

Because **core business logic should not change** when:

- You replace the database
 - You switch UI
 - You change logging framework
 - You migrate from ASP.NET Core 7 → .NET 8 → .NET 10
- Your **domain remains untouched**.

🧠 Why Clean Architecture Is Used in Real Projects?

- ✓ Easy to test
- ✓ Easy to change database
- ✓ Easy to migrate UI (Web API → UI → Mobile App)
- ✓ Code separation improves maintenance
- ✓ Clear responsibility boundaries

☀️ Benefits of Clean Architecture

Benefit	Explanation
Testability	Domain and Application layers are easy to unit test.
Flexibility	Easy to replace DB, UI, frameworks.
Maintainability	Clear separation of responsibilities.
Scalability	Supports microservices and modular architectures.
Long-term sustainability	Reduces technical debt.

Summary

Layer	Responsibility	Should Know About
Domain	Entities + Business Rules	Nothing outside itself
Application	Use cases + Orchestration	Domain
Infrastructure	EF Core, DB, Email, Files	Application
Presentation	API controllers	Application